

Proyecto Backend

Sistema de Reservas de Club Deportivo

Fecha de entrega: A definir

Modalidad: Proyecto grupal (desde 6 integrantes)

Entrega: Link a repositorio GitHub público

Contexto del negocio

El Club Deportivo Encuentro dispone de canchas de fútbol, tenis y pádel que sus socios pueden reservar. Actualmente, el personal recibe las solicitudes por teléfono y mensajes, y registra los turnos en una planilla compartida.

Este procedimiento genera reservas superpuestas, dificultades para consultar los horarios disponibles y cancelaciones que no se reflejan correctamente. Además, los cambios de tarifas dificultan reconstruir el importe acordado para una reserva anterior.

El club solicita una API que centralice la información de sus canchas y socios, permita gestionar reservas y conserve el historial de las operaciones.

Alcance inicial. Se trabajará con un único club. No se administrarán cuotas sociales, pagos, torneos ni inscripciones a clases.

1. Objetivo

Desarrollar una API REST en Python y Flask que permita administrar canchas y socios, consultar disponibilidad, registrar reservas y actualizar sus estados.

La solución deberá validar los horarios solicitados, evitar superposiciones, calcular el importe de cada reserva y ofrecer listados con filtros y paginación.

El trabajo comprende únicamente el backend. No se requiere desarrollar una interfaz gráfica.

2. Contexto

¿Qué es una cancha?

Es un espacio físico del club que puede reservarse para practicar un deporte. Para simplificar, cada cancha estará asociada a un único deporte y se reservará completa: no se administrarán cupos individuales ni subdivisiones.

¿Qué es un socio?

Es una persona registrada en el club. Cada reserva tendrá un socio responsable; no será necesario registrar a sus acompañantes.

¿Qué es una reserva?

Es la asignación de una cancha a un socio para una fecha y un intervalo horario determinados. Conserva el estado de la solicitud y el importe calculado al registrarla.

¿Qué significa que una cancha esté disponible?

Significa que está habilitada para nuevas reservas y no tiene una reserva confirmada que se superponga con el intervalo consultado. Una cancha habilitada puede estar ocupada en algunos horarios y disponible en otros.

3. Consideraciones generales

Alcance y contrato

La API recibirá y devolverá datos en JSON. La información deberá persistir en una base de datos SQL y el contrato se documentará en swagger.yaml.

No se requiere autenticación ni envío de notificaciones. Se utilizarán datos ficticios. Los deportes se cargarán mediante un script inicial (init_db.sql); no se exige implementar su alta, modificación o eliminación.

Fechas y horarios

- El club atenderá todos los días de 08:00 a 23:00, sin excepciones por feriados.
- Las reservas durarán entre una y tres horas completas, comenzarán y terminarán en horas en punto y no podrán atravesar la medianoche.
- La API recibirá fecha y hora en formato ISO 8601 con desplazamiento fijo GMT-3: YYYY-MM-DDTHH:MM:SS.ffffff-03:00 (6 dígitos de fracción de segundo). Todos los valores se interpretarán en GMT-3, sin conversión de zona horaria.

- Solo se podrán crear reservas cuyo inicio sea posterior al momento actual.
- Deberá cumplirse $\text{fecha_hora_inicio} < \text{fecha_hora_fin}$ y el intervalo completo deberá quedar dentro del horario del club.

Disponibilidad y superposiciones

No se permitirán reservas confirmadas superpuestas para una misma cancha. Tampoco se permitirá que un socio tenga reservas confirmadas superpuestas, aunque sean en canchas distintas.

Por ejemplo, si una cancha está reservada de 18:00 a 20:00, no podrá reservarse de 19:00 a 21:00, pero sí de 20:00 a 21:00. También deberán rechazarse intervalos idénticos y aquellos que contengan o estén contenidos en otra reserva.

La consulta de disponibilidad será únicamente informativa y no creará ni retendrá una reserva. Al registrar una reserva, la API deberá volver a verificar que la cancha y el socio continúen disponibles para el intervalo solicitado.

Importes

- Los precios se expresarán como enteros en centavos. Por ejemplo, 1000000 representa \$10.000,00.
- La tarifa por hora debe ser mayor que cero.
- El servidor calculará el total multiplicando la cantidad de horas por la tarifa vigente de la cancha.
- La reserva conservará esa tarifa y ese total, aunque posteriormente cambie el precio de la cancha.
- El cliente de la API no podrá establecer identificadores generados, tarifas históricas, totales ni el estado inicial.

Estados y cancelaciones

Toda reserva se creará en estado confirmada.

Estado actual	Estado solicitado	Condición
confirmada	cancelada	El horario de inicio todavía no llegó
confirmada	finalizada	Se alcanzó o superó el horario de finalización
cancelada	Otro estado	No permitido
finalizada	Otro estado	No permitido

- Repetir el estado actual devolverá éxito sin modificar la reserva.
- Los cambios serán explícitos: no se requieren tareas automáticas.

- Una cancelación libera el horario, pero no elimina el registro. No se permite reactivar reservas canceladas.
- Desactivar una cancha o un socio impedirá crear nuevas reservas asociadas, pero no cancelará ni invalidará automáticamente las existentes.
- En el alcance obligatorio no se modificarán la cancha, el socio ni los horarios de una reserva ya creada. Para corregirla se deberá cancelar, cuando corresponda, y registrar una nueva.

4. API a implementar

Convenciones comunes

- Los identificadores serán enteros positivos.
- Los filtros se combinarán con criterio AND y las búsquedas por nombre serán parciales, sin distinguir mayúsculas de minúsculas.
- Los filtros booleanos admitirán únicamente true o false.
- Los cuerpos deberán ser objetos JSON con los tipos indicados.
- Se rechazarán campos y parámetros desconocidos, valores inválidos y cuerpos vacíos en actualizaciones.
- Las creaciones exitosas responderán 201. Las consultas y actualizaciones exitosas responderán 200.
- Las respuestas de listado incluirán paginación con `_limit`, `_offset` y navegación HATEOAS (`_first`, `_prev`, `_next`, `_last`).

Endpoints obligatorios

Deportes

GET /deportes

Consultar los deportes precargados. No requiere paginación.

Canchas

GET /canchas

Listar las canchas con paginación.

- Filtros opcionales: *id_deporte*, *nombre*, *techada* y *activa*.
- Sin filtros se incluirán tanto las canchas activas como las inactivas.

POST /canchas

Crear una cancha.

- Campos obligatorios: *nombre*, *id_deporte* y *precio_hora*.
- Campos opcionales: *techada*, con valor predeterminado *false*, y *activa*, con valor predeterminado *true*.
- El nombre no podrá quedar vacío después de quitar espacios en sus extremos.
- El deporte deberá existir y el precio deberá ser un entero positivo.

GET /canchas/{id}

Obtener los datos de una cancha. Su estado *activa* no indica que esté libre en todos los horarios.

PATCH /canchas/{id}

Actualizar parcialmente una cancha.

- Campos editables: *nombre*, *precio_hora*, *techada* y *activa*.
- Los campos omitidos conservarán su valor y se aplicarán las validaciones del alta.
- El deporte asociado no se modificará una vez creada la cancha.
- Cambiar el precio no alterará los importes de reservas existentes.

DELETE /canchas/{id}

Eliminar una cancha únicamente si no tiene ninguna reserva asociada, independientemente de su estado.

- Si tiene reservas, responder 409; podrá desactivarse mediante PATCH.
- La eliminación exitosa responderá 204, sin cuerpo.

GET /canchas/disponibles

Consultar canchas activas libres durante todo un intervalo.

- Parámetros obligatorios: *fecha*, *hora_inicio* y *hora_fin*.
- Filtros opcionales: *id_deporte* y *techada*.
- El resultado tendrá paginación.
- El intervalo deberá cumplir las mismas reglas de fecha, horario y duración que una reserva nueva.
- Si no hay canchas libres, responder 200 con un arreglo vacío.
- Esta consulta informa disponibilidad de las canchas. La habilitación y la agenda del socio se validarán al crear la reserva.

Socios

GET /socios

Listar socios con paginación. Filtros opcionales: *nombre* y *activo*.

POST /socios

Registrar un socio con *nombre* y *email*, ambos obligatorios.

- El servidor asignará *activo*: true.
- El nombre no podrá quedar vacío.
- El correo deberá tener un formato válido y almacenarse en minúsculas, sin espacios en sus extremos.
- Un correo ya registrado producirá 409, incluso si el socio existente está inactivo.

GET /socios/{id}

Consultar los datos de un socio.

PATCH /socios/{id}

Actualizar parcialmente *nombre*, *email* o *activo*, respetando las validaciones del alta y la unicidad del correo.

- Los campos omitidos conservarán su valor.
- No se requiere un endpoint de eliminación de socios.

Reservas

GET /reservas

Listar reservas con paginación.

- Filtros opcionales: *id_cancha*, *id_socio*, *estado*, *fecha_desde* y *fecha_hasta*.
- El rango se aplicará al día de utilización de la cancha, con ambos extremos incluidos.
- Podrá enviarse un solo extremo; si se envían ambos, deberá cumplirse *fecha_desde* <= *fecha_hasta*.
- Se permitirá consultar reservas pasadas.

POST /reservas

Crear una reserva con *id_socio*, *id_cancha*, *fecha_hora_inicio* y *fecha_hora_fin*, todos obligatorios.

```
{
  "id_socio": 1,
  "id_cancha": 2,
  "fecha_hora_inicio": "2026-10-15T18:00:00.000000-03:00",
```

```
{
  "fecha_hora_fin": "2026-10-15T20:00:00.000000-03:00"
}
```

- La API deberá verificar que el socio y la cancha existan y estén activos, validar el intervalo y comprobar que no haya superposiciones para ninguno de ellos.
- El servidor asignará el estado confirmada, conservará la tarifa vigente y calculará el total.
- Para dos horas a 1000000 centavos por hora, el total será 2000000 centavos.
- Si alguna validación falla, no deberá guardarse la reserva.
- La fecha del ejemplo es ilustrativa: al probarlo deberá utilizarse una fecha futura.

GET /reservas/{id}

Obtener todos los campos de una reserva, incluidos el estado, la tarifa histórica y el importe total.

PUT /reservas/{id}/estado

Establecer el estado de una reserva respetando las transiciones y restricciones temporales de la sección 3.

```
{
  "estado": "cancelada"
}
```

- Un estado desconocido producirá 400.
- Una transición no permitida, o solicitada fuera del momento permitido, producirá 409.
- La respuesta exitosa incluirá la reserva con su estado actual.

5. Paginación

Los endpoints de listado admitirán `_limit` y `_offset`.

Parámetro	Valor predeterminado	Restricción
<code>_limit</code>	10	Entero entre 1 y 100
<code>_offset</code>	0	Entero mayor o igual a cero

- Los resultados se filtrarán antes de paginar y se ordenarán por id ascendente.
- La respuesta incluirá los datos paginados bajo una clave descriptiva (canchas, socios o reservas) y un objeto `_links` con los enlaces HATEOAS `_first`, `_prev`, `_next` y `_last`.

6. Entregables

El repositorio deberá incluir:

- Código fuente de la aplicación, dependencias en requirements.txt y configuración de ejemplo sin credenciales reales.
- Contrato swagger.yaml.
- Scripts de creación de la base de datos y datos ficticios de prueba, incluidos los deportes iniciales.
- Un README.md con integrantes, versiones utilizadas, pasos de instalación y ejecución, configuración, ejemplos de solicitudes y supuestos adoptados.

Condición de entrega. La aplicación deberá poder ejecutarse desde una copia nueva del repositorio siguiendo el README.

7. Requisitos técnicos

Lenguaje: Python 3.

Framework: Flask.

Persistencia: Base de datos SQL, preferentemente MySQL.

Las rutas HTTP, las validaciones, las reglas de negocio y el acceso a datos deberán organizarse con responsabilidades diferenciadas. No se impone una estructura única de carpetas ni el uso de programación orientada a objetos.

Las consultas deberán incorporar los datos recibidos mediante parámetros. La configuración sensible no deberá publicarse en el repositorio y reiniciar la aplicación no deberá borrar la información.

8. Evaluación

Se considerarán el funcionamiento de los endpoints, la correspondencia entre el contrato y la implementación, el modelo de datos, los filtros, la paginación, el tratamiento de errores y la claridad del código.

Las pruebas deberán contemplar, como mínimo:

- Creación válida de canchas, socios y reservas; cálculo correcto del importe y conservación de la tarifa histórica.
- Rechazo de superposiciones para la cancha y para el socio, incluidos intervalos iguales, contenidos y contenedores; aceptación de reservas consecutivas.
- Rechazo de horarios inválidos, reservas que no sean futuras, referencias inexistentes y entidades inactivas, sin guardar registros incompletos.

- Cancelación que libera el horario, finalización, transiciones rechazadas y repetición del estado actual.
- Restricción de eliminación de canchas con reservas y combinación de filtros con paginación.

Las funcionalidades opcionales no reemplazan los requisitos obligatorios.

9. Nota final

Problema central. Gestionar la disponibilidad de un recurso a lo largo del tiempo. No alcanza con guardar una reserva: la API debe comprobar que puede aceptarla y mantener coherencia entre horarios, estados e importes.

No se requiere implementar un sistema comercial completo. El alcance está limitado a las operaciones definidas en este enunciado.

10. Extensión opcional

Bloqueos por mantenimiento

Incorporar una entidad Bloqueo con cancha, fecha, horario de inicio, horario de fin y motivo, junto con operaciones para crear, consultar y eliminar bloqueos.

- Un bloqueo impedirá realizar reservas durante ese intervalo y deberá considerarse en la consulta de disponibilidad.
- No podrá crearse si se superpone con una reserva confirmada o con otro bloqueo de la misma cancha.
- Quitar un bloqueo liberará únicamente la restricción que ese bloqueo generaba.
- Los bloqueos deberán comenzar en el futuro, permanecer dentro de un mismo día y respetar el horario del club y las horas en punto.
- Podrán durar más de tres horas para permitir mantenimiento durante toda la jornada.

Endpoints propuestos:

- **POST** /bloqueos
- **GET** /bloqueos
- **DELETE** /canchas/{id}

Reservas recurrentes

Incorporar la posibilidad de crear una serie de reservas semanales para la misma cancha, el mismo socio y el mismo intervalo horario.

La solicitud deberá indicar la fecha de la primera reserva y una cantidad de semanas entre 2 y 12. La API generará una reserva para cada semana, siempre en el mismo día y horario.

Antes de crear la serie, se deberán validar todas las reservas involucradas:

- El socio y la cancha deberán existir y estar activos.
- Todas las fechas deberán ser futuras.
- Los intervalos deberán respetar el horario y la duración permitidos.
- Ninguna reserva podrá superponerse con otra reserva confirmada de la cancha o del socio.

La creación deberá ser una operación completa: si alguna de las fechas no está disponible, no se guardará ninguna reserva de la serie. En ese caso, la API responderá 409 e informará cuáles son las fechas que presentan conflictos.

Si todas las fechas son válidas, se crearán las reservas y se responderá 201 con el listado completo.

Endpoints propuesto:

- **POST /reservas/recurrentes**